```
def build_tree(X, y, depth=0, max_depth=5):
    num_samples, num_features = X.shape
    num_labels = len(np.unique(y))
    # Base cases: Pure node, max depth, or no more samples
    if num_labels == 1 or depth >= max_depth or num_samples < 2
        leaf_value = Counter(y).most_common(1)[0][0]
        return {'Value': leaf_value}

    idx, thr = get_best_split(X, y)
    if idx is None:
        return {'value': Counter(y).most_common(1)[0][0]}
    # Split the data & resources
    left_indices = X[:, idx] <= thr

left_node = build_tree(X[left_indices], y[left_indices], depth+1, max_depth)
right_node = build_tree(X[~left_indices], y[~left_indices], depth+1, max_depth)

return {
    'feature_idx': idx,
    'threshold': thr,
    'left': left_node,
    'right': right_node
}

def predict_one(tree, sample):
    if 'value' in tree:
        return tree['value']
    if sample[tree['feature_idx']] <= tree['threshold']:
        return predict_one(tree['left'], sample)
    else:
        return predict_one(tree['right'], sample)

iris = load_iris()
X, y = iris.data, iris.target
my_tree = build_tree(X, y, max_depth=3)

Prediction = Predict_one(my_tree, X[sample_index])
Print(f"Actual: {iris.target_name
```

```
import numpy as np
from sklear.datasets import load_iris
from collections import Counter

# 1. Calculate Gini Impurity

def calculate_gini(x):
    m = len(y)
    if m == 0: return 0
    counts = Counter(y)
    # Gini = 1 - sum(p_i^2)
    return 1.0 - sum((count/m) ** 2 for count in
                                    counts.values))

def get_best_split(X, y):
    m, n = X.shape
    if m <= 1: return None, None
    best_gini = 999
    best_idx, best_thr = None, None

    for idx in range(n): Loop through features
        thresholds = np.unique(X[:, idx])
        for thr in thresholds:- # Loops through unique
            # Create a mask for binary split
            left_indices = X[:, idx] <= thr
            y_left, y_right = y[left_indices], y[~left_indices]
            if len(y_left) == 0 or len(y_right) == 0
            Continue
            n_l, n_r = len(y_left), len(y_right)
            gini = (n_l/m) * Calculate_gini(y_left) + (n_r/m)
                                                * Calculate_gini
                                                (y_right)
            if gini < best_gini
                best_gini = gini
                best_idx = idx
                best_thr = thr
    return best_idx, best_thr
```